

LAPORAN TUGAS UJIAN AKHIR SEMESTER
IMPLEMENTASI ALGORITMA A* DAN DIJKSTRA PADA SISTEM NAVIGASI
KAMPUS UNIVERSITAS BENGKULU BERBASIS OPENSTREETMAP



Disusun Oleh:

Nama : Muhammad Yusuf Arrayyan

NPM : G1A024009

Kelas : A (Informatika)

Mata Kuliah:

Kecerdasan Buatan

Dosen Pengampu:

Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP.

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS BENGKULU

2026

DAFTAR ISI

BAB I.....	3
PENDAHULUAN	3
1.1 Latar Belakang.....	3
1.2 Rumusan Masalah	4
1.3 Tujuan.....	4
BAB II	5
LANDASAN TEORI	5
2.1 Kecerdasan Buatan (<i>Artificial Intelligence</i>).....	5
2.2 Algoritma Dijkstra.....	5
2.3 Algoritma A (A-Star) dan Heuristik	5
2.4 Perbandingan Algoritma Dijkstra dan A*	6
2.5 OpenStreetMap dan Sistem Informasi Geografis.....	6
BAB III.....	7
METODOLOGI DAN PERANCANGAN SISTEM.....	7
3.1 Gambaran Umum Sistem.....	7
3.2 Arsitektur Sistem	7
3.3 Representasi Graf Area Kampus Universitas Bengkulu.....	8
3.4 Alur Kerja Sistem	8
BAB IV	9
IMPLEMENTASI DAN ANALISIS SISTEM	9
4.1 Implementasi Antarmuka Pengguna Antarmuka pengguna (<i>User Interface</i>)	9
4.2 Implementasi Pencarian Jalur.....	9
4.3 Analisis dan Perbandingan Kinerja Algoritma	10
4.4 Pengujian Fungsionalitas Sistem	11
BAB V	12
PENUTUP.....	12
5.1 Kesimpulan.....	12
5.2 Saran	12
DAFTAR PUSTAKA.....	13
LAMPIRAN.....	14

BAB I PENDAHULUAN

1.1 Latar Belakang

Mobilitas dan efisiensi waktu merupakan faktor krusial dalam aktivitas akademik di lingkungan pendidikan berskala besar. Universitas Bengkulu, sebagai salah satu institusi pendidikan utama di Provinsi Bengkulu, memiliki area kampus yang sangat luas dengan tata letak infrastruktur yang tersebar meliputi gedung fakultas, pusat administrasi, laboratorium, hingga fasilitas umum lainnya. Kompleksitas tata letak kampus ini sering kali membingungkan individu yang belum terbiasa dengan denah lokasi, khususnya mahasiswa baru dan tamu universitas. Mahasiswa sering kali dihadapkan pada jadwal perkuliahan yang padat dengan jeda waktu yang sempit untuk berpindah antar gedung, sehingga efisiensi rute pergerakan menjadi sangat penting. Selama ini, pencarian lokasi umumnya masih mengandalkan memori spasial, arahan dari orang lain, atau papan petunjuk fisik yang jumlahnya terbatas.

Di sisi lain, penggunaan aplikasi peta komersial seperti Google Maps kerap kali tidak memberikan solusi yang optimal untuk lingkungan kampus. Sistem tersebut umumnya dirancang untuk navigasi kendaraan di jaringan jalan raya makro dan sering kali tidak mendata jalan setapak pedestrian, koridor antar gedung, atau rute internal kampus secara detail. Oleh karena itu, diperlukan inovasi menuju konsep *smart campus* berupa sistem navigasi spasial mandiri yang dirancang khusus untuk memetakan rute internal kampus dan mampu menghitung jalur terpendek secara akurat. Dalam disiplin ilmu Kecerdasan Buatan (*Artificial Intelligence*), tantangan untuk menemukan rute paling efisien dari satu titik ke titik lain dengan mempertimbangkan berbagai rintangan dikenal sebagai *Shortest Path Problem*. Untuk memecahkan masalah ini pada data spasial dunia nyata yang direpresentasikan dalam bentuk graf (terdiri dari *node* dan *edge*), terdapat berbagai metode algoritma pencarian. Dua algoritma fundamental yang paling menonjol dan sering dijadikan standar pengujian dalam komputasi rute adalah algoritma Dijkstra dan algoritma A* (*A-Star*).

Algoritma Dijkstra beroperasi dengan prinsip *greedy*, mengeksplorasi seluruh simpul yang tersedia secara merata dari titik awal ke segala arah. Algoritma ini sangat andal dan menjamin penemuan rute terpendek yang absolut, namun memiliki kelemahan dalam hal beban komputasi ketika dihadapkan pada graf yang sangat besar. Sementara itu, algoritma A* menawarkan pendekatan yang lebih cerdas dan terarah dengan mengkombinasikan biaya aktual yang telah ditempuh dengan fungsi heuristik—sebuah estimasi matematis (seperti jarak garis lurus) menuju titik tujuan. Penambahan kecerdasan heuristik ini memungkinkan A*

untuk memfokuskan arah pencarian secara langsung ke target, sehingga secara teoritis dapat memangkas waktu komputasi dan memori secara signifikan tanpa mengorbankan akurasi hasil.

Berangkat dari urgensi tersebut, proyek ini bertujuan untuk merancang dan membangun sebuah sistem navigasi spasial berbasis web untuk lingkungan Universitas Bengkulu dengan memanfaatkan data terbuka dari *OpenStreetMap*. Berbeda dengan sistem navigasi tunggal pada umumnya, sistem ini mengimplementasikan algoritma Dijkstra dan A* secara berdampingan. Pendekatan ini tidak hanya menghasilkan purwarupa aplikasi pemetaan yang interaktif, dapat diakses tanpa instalasi (*browser-based*), dan fungsional, tetapi juga membuka ruang untuk melakukan analisis komparatif yang komprehensif. Melalui sistem ini, efisiensi kinerja komputasi dan jumlah langkah eksplorasi antara algoritma Dijkstra dan A* dapat diuji, dibandingkan, dan dibuktikan secara langsung ketika dihadapkan pada kasus pemetaan tata letak dunia nyata.

1.2 Rumusan Masalah

Berdasarkan pemaparan latar belakang di atas, rumusan masalah dalam penelitian ini adalah:

1. Bagaimana merancang dan mengimplementasikan sistem navigasi interaktif berbasis web untuk area Universitas Bengkulu dengan memanfaatkan data spasial dari *OpenStreetMap*?
2. Bagaimana mekanisme penerapan algoritma Dijkstra dan algoritma A* dalam mengeksekusi pencarian rute terpendek pada struktur graf jaringan kampus?
3. Bagaimana perbandingan efektivitas dan efisiensi kinerja komputasi antara algoritma Dijkstra dan algoritma A* ketika dijalankan pada *dataset* peta kampus yang sama?

1.3 Tujuan

Adapun tujuan yang ingin dicapai melalui proyek dan laporan ini adalah:

1. Menghasilkan purwarupa aplikasi navigasi kampus (*pathfinding*) yang fungsional dan responsif untuk membantu pemetaan rute di dalam area Universitas Bengkulu.
2. Mengimplementasikan logika algoritma Dijkstra dan A* secara langsung ke dalam sistem berbasis koordinat geografis nyata.
3. Mengukur dan menganalisis secara objektif perbandingan performa antara algoritma Dijkstra (sebagai *blind search*) dan A* (sebagai *heuristic search*) dalam studi kasus ruang lingkup kampus.

BAB II

LANDASAN TEORI

2.1 Kecerdasan Buatan (*Artificial Intelligence*)

Kecerdasan Buatan atau *Artificial Intelligence* (AI) merupakan cabang ilmu komputer yang berfokus pada pengembangan sistem yang mampu meniru kemampuan berpikir manusia dalam menyelesaikan suatu permasalahan. Menurut Russell dan Norvig (2021), *Artificial Intelligence* merupakan studi mengenai agen cerdas yang mampu memahami lingkungan di sekitarnya dan mengambil tindakan untuk mencapai tujuan tertentu. Dalam konteks pemetaan spasial, penerapan agen cerdas ini difokuskan pada penyelesaian *Shortest Path Problem*, yaitu proses pencarian jalur navigasi paling efisien antar dua titik dengan mempertimbangkan bobot atau biaya perjalanan.

2.2 Algoritma Dijkstra

Algoritma Dijkstra merupakan algoritma pencarian jalur terpendek pada graf berbobot yang dikembangkan oleh Edsger W. Dijkstra. Algoritma ini dirancang untuk menyelesaikan masalah lintasan terpendek pada sebuah graf berbobot non-negatif. Prinsip kerja Dijkstra didasarkan pada pendekatan *greedy*, di mana algoritma ini secara sistematis mengeksplorasi semua simpul (*node*) tetangga dari titik awal, menghitung jarak kumulatif, dan selalu memilih *node* dengan biaya kumulatif terkecil yang belum diproses.

Fungsi evaluasi pada algoritma Dijkstra murni didasarkan pada perhitungan biaya aktual, yaitu akumulasi jarak yang telah ditempuh dari titik awal menuju ke simpul yang sedang dievaluasi. Algoritma ini sama sekali tidak menggunakan perkiraan heuristik mengenai jarak yang tersisa menuju tujuan akhir. Keunggulan utama algoritma Dijkstra adalah jaminan mutlak untuk menemukan rute paling optimal. Namun, karena mengeksplorasi ke segala arah secara menyeluruh, algoritma ini memiliki kelemahan dalam hal efisiensi beban komputasi ketika dihadapkan pada graf jaringan yang sangat besar.

2.3 Algoritma A (A-Star) dan Heuristik

Algoritma A* merupakan algoritma pencarian heuristik yang diperkenalkan oleh Hart, Nilsson, dan Raphael pada tahun 1968. Algoritma ini dirancang untuk menemukan jalur terpendek dengan memanfaatkan biaya aktual perjalanan dan estimasi biaya menuju tujuan. Pendekatan ini mengubah strategi pencarian dari pencarian buta menjadi pencarian terarah (*informed search*).

Algoritma A* menggunakan mekanisme evaluasi yang menggabungkan dua elemen utama. Elemen pertama adalah biaya aktual perjalanan dari titik awal ke simpul saat ini.

Elemen kedua adalah nilai heuristik, yang merupakan estimasi biaya atau perkiraan jarak langsung dari simpul yang sedang dievaluasi tersebut menuju ke titik tujuan. Kunci keberhasilan A* terletak pada fungsi heuristik ini, karena ia membantu algoritma mempersempit ruang pencarian sehingga proses pencarian menjadi lebih efisien. Penggunaan heuristik ini memungkinkan algoritma memprioritaskan pengecekan pada simpul yang secara geometris mengarah mendekati tujuan (Hart et al., 1968).

2.4 Perbandingan Algoritma Dijkstra dan A*

Dalam penentuan rute terdekat, algoritma Dijkstra dan A* memiliki karakteristik operasional yang sangat berbeda meskipun keduanya sama-sama digunakan untuk menyelesaikan masalah navigasi (Prasetyo, 2019). Algoritma Dijkstra bekerja secara ekspansif dengan menjelajahi semua simpul secara menyeluruh berdasarkan prioritas biaya aktual perjalanan tanpa adanya panduan arah menuju titik tujuan. Karakteristik komputasi ini membuat Dijkstra sangat andal dalam menjamin penemuan rute yang optimal pada sebuah graf berbobot, namun tingkat efisiensi pencariannya tergolong lebih rendah karena algoritma akan mengeksplorasi secara merata ke segala arah. Sebaliknya, algoritma A* dirancang untuk menyempurnakan pendekatan komputasi tersebut dengan menambahkan fungsi heuristik di atas logika dasar pencarian. Penambahan kecerdasan heuristik ini membuat arah eksplorasi algoritma A* menjadi jauh lebih terfokus secara spesifik menuju target lokasi akhir. Pendekatan heuristik ini memberikan keunggulan komparatif pada algoritma A* berupa kemampuannya dalam menghasilkan jalur optimal dengan waktu pencarian yang relatif jauh lebih cepat apabila dibandingkan dengan algoritma pencarian tanpa heuristik, seperti Dijkstra (Hart et al., 1968).

2.5 OpenStreetMap dan Sistem Informasi Geografis

Sistem Informasi Geografis (SIG) memungkinkan pengguna untuk mengumpulkan, mengelola, menganalisis, dan menampilkan data yang memiliki informasi geografis (Longley dkk., 2015). Dalam pengembangan aplikasi web ini, *OpenStreetMap* (OSM) digunakan sebagai proyek pemetaan terbuka untuk sumber data peta dasar yang menampilkan lokasi gedung dan tata letak infrastruktur. Integrasi data OSM menggunakan pustaka antarmuka seperti *Leaflet.js* memungkinkan representasi dunia nyata divisualisasikan secara langsung di dalam peramban web (*browser*), sehingga memudahkan sistem dalam melakukan interaksi pemetaan.

BAB III METODOLOGI DAN PERANCANGAN SISTEM

3.1 Gambaran Umum Sistem

Sistem navigasi kampus Universitas Bengkulu yang dikembangkan dalam proyek ini merupakan sebuah perangkat lunak terapan berbasis web interaktif yang dirancang khusus untuk menjembatani permasalahan orientasi spasial di lingkungan akademis. Berkat arsitekturnya yang berbasis web, aplikasi ini menawarkan tingkat aksesibilitas yang tinggi, di mana sivitas akademika maupun tamu pengunjung dapat mengakses layanan navigasi secara langsung melalui peramban (*browser*) di berbagai macam perangkat pintar tanpa memerlukan proses unduhan atau instalasi aplikasi tambahan.

Secara fundamental, sistem ini dirancang untuk memfasilitasi pencarian rute terpendek antar berbagai titik lokasi strategis dan gedung fakultas di dalam area kampus. Guna merepresentasikan kondisi geografis yang nyata dengan tingkat akurasi tinggi, sistem ini memanfaatkan integrasi basis data pemetaan spasial terbuka dari *OpenStreetMap* (OSM). Penggunaan data spasial dari OSM memberikan keunggulan komparatif karena memuat detail topologi jaringan jalan internal kampus, mulai dari jalan raya utama kendaraan hingga koridor atau jalan setapak pedestrian yang umumnya tidak terindeks secara detail pada layanan peta.

Nilai kebaruan dan fitur utama yang menjadi sorotan sentral dari rotoc ini adalah penerapan implementasi ganda (*dual-implementation*) dari algoritma pencarian rute berbasis Kecerdasan Buatan, yaitu algoritma Dijkstra dan algoritma A* (*A-Star*). Implementasi komparatif ini tidak hanya berfungsi sebagai mesin pencari jalur utama (*pathfinding engine*), tetapi juga sebagai protokol pengujian analitis untuk membuktikan teori komputasi dalam studi kasus tata letak dunia nyata.

3.2 Arsitektur Sistem

Pengembangan navigasi ini mengadopsi arsitektur klien-peladen (*client-server architecture*) yang memisahkan antara antarmuka pengguna dan logika pemrosesan data.

1. **Bagian Klien (*Frontend*):** Dibangun menggunakan rotoc pemrograman *markup* dan skrip web standar yang dikombinasikan dengan rotoco *Tailwind CSS* untuk desain antarmuka yang rotocol e dan modern. Pustaka *Leaflet.js* diintegrasikan sebagai komponen utama untuk merender ubin peta (*tile map*) dari peladen *OpenStreetMap* dan memanipulasi objek spasial (seperti penanda rotoc dan garis rute) di peramban web.
2. **Bagian Peladen (*Backend*):** Bertugas sebagai pusat komputasi matematis. Peladen ini mengelola data graf internal kampus dan mengeksekusi logika algoritma Dijkstra serta

A* saat menerima instruksi permintaan (*request*) dari sisi klien melalui antarmuka pemrograman aplikasi (*REST API*).

3.3 Representasi Graf Area Kampus Universitas Bengkulu

Agar algoritma kecerdasan buatan dapat memproses data peta, tata letak geografis Universitas Bengkulu harus dikonversi terlebih dahulu ke dalam bentuk model matematika yang disebut Graf. Proses pemodelan ini melibatkan dua elemen utama:

1. **Simpul (*Node*):** Merepresentasikan titik-titik koordinat penting di dalam kampus, seperti gerbang masuk, persimpangan jalan pedestrian, bangunan fakultas, dan fasilitas umum lainnya. Setiap simpul menyimpan atribut identitas berupa garis lintang (*latitude*) dan garis bujur (*longitude*).
2. **Sisi (*Edge*):** Merepresentasikan jalur atau jalan fisik yang menghubungkan antar simpul. Setiap sisi memiliki bobot (*weight*) yang dikalkulasi berdasarkan jarak jarak tempuh melengkung di permukaan bumi antar koordinat geografis.

3.4 Alur Kerja Sistem

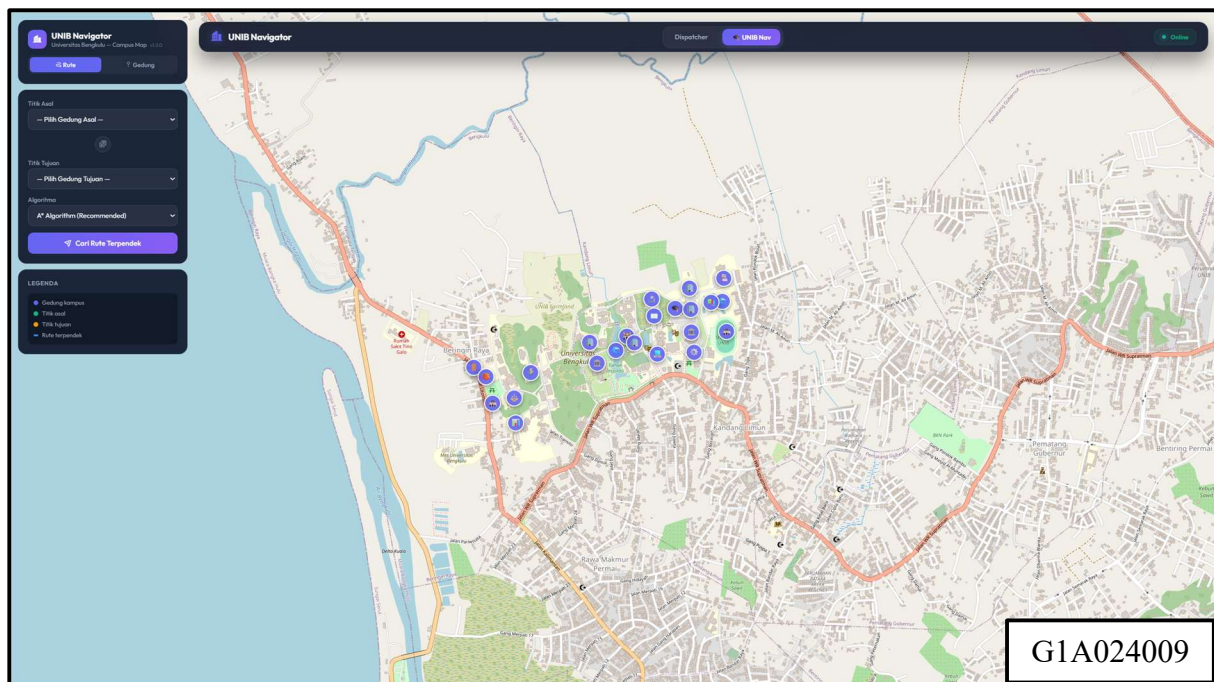
Sistem memiliki alur kerja sekuensial yang dimulai dari interaksi pengguna hingga visualisasi rute. Proses tersebut dijabarkan sebagai berikut:

1. Pengguna membuka protocol melalui peramban web. Sistem akan memuat antarmuka peta Universitas Bengkulu secara interaktif dan menampilkan penanda rotoc rotoc-gedung utama.
2. Pengguna menentukan parameter navigasi dengan memilih Titik Awal dan Titik Tujuan melalui panel kendali yang tersedia pada antarmuka.
3. Sistem sisi klien mengirimkan data koordinat yang dipilih ke rotoc sisi peladen melalui protocol komunikasi web.
4. Sistem peladen menerima data tersebut dan menjalankan algoritma Dijkstra serta A* secara komparatif pada graf jalan kampus. Algoritma A* menggunakan formula jarak garis lurus (*Haversine*) sebagai nilai heuristiknya.
5. Sistem mencatat matriks performa dari kedua algoritma, meliputi total jarak rute terpendek yang dihasilkan, jumlah simpul yang dievaluasi, dan waktu eksekusi komputasi (dalam satuan milidetik).
6. Hasil kalkulasi dikembalikan ke sisi klien. Sistem kemudian menggambar garis rute perjalanan (*polyline*) di atas lapisan peta dan menampilkan rangkuman perbandingan metrik kinerja algoritma pada panel informasi pengguna.

BAB IV IMPLEMENTASI DAN ANALISIS SISTEM

4.1 Implementasi Antarmuka Pengguna Antarmuka pengguna (*User Interface*)

pada sistem navigasi kampus Universitas Bengkulu ini dirancang dengan mengedepankan prinsip *clean design* dan kemudahan navigasi (*user-friendly*). Dibangun menggunakan kombinasi HTML, JavaScript, dan Tailwind CSS, halaman utama sistem langsung menyajikan kanvas peta interaktif berskala penuh yang ditenagai oleh pustaka *Leaflet.js* dan data ubin peta dari *OpenStreetMap*. Pada tampilan awal, sistem menampilkan berbagai penanda (*marker*) yang merepresentasikan titik-titik lokasi penting di area kampus, seperti dekanat, rektorat, dan fasilitas umum lainnya.



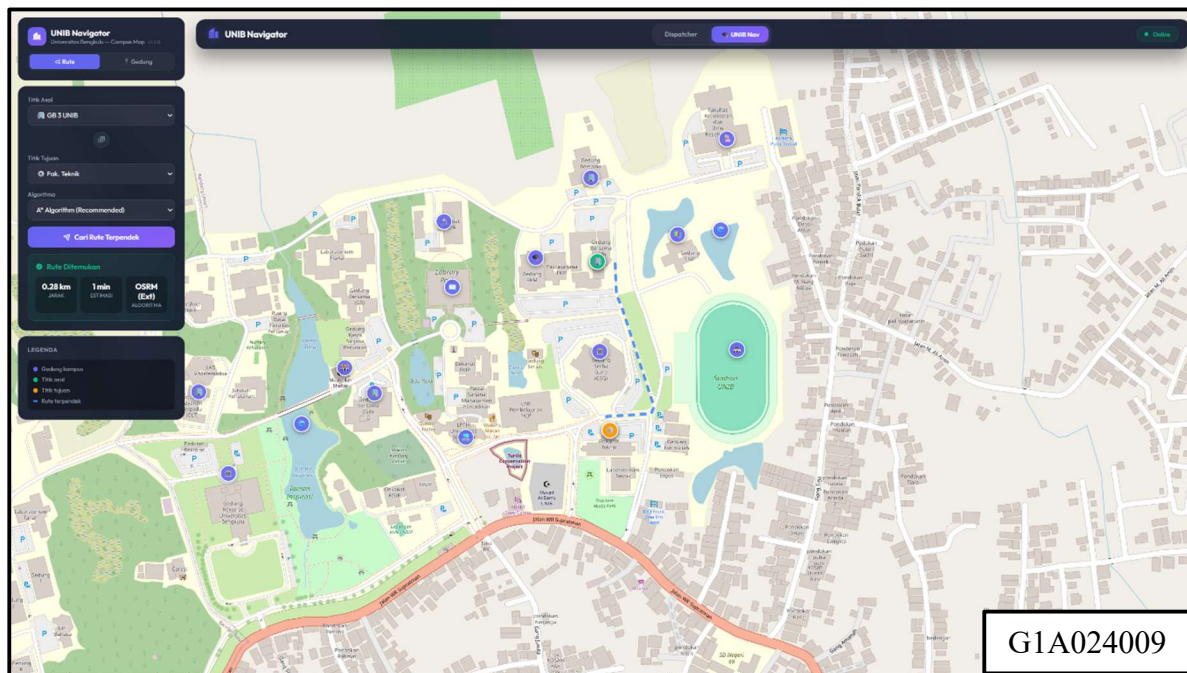
Gambar 1.1 Tampilan Antarmuka Utama Sistem Navigasi Kampus

Di sisi kanvas peta, terdapat sebuah panel navigasi yang berfungsi sebagai pusat kendali pengguna. Melalui panel ini, pengguna dapat memilih lokasi awal dan lokasi tujuan dari daftar *drop-down* yang telah diintegrasikan dengan basis data koordinat kampus. Tersedia pula tombol eksekusi untuk memicu kalkulasi algoritma kecerdasan buatan dalam mencari rute terpendek.

4.2 Implementasi Pencarian Jalur

Proses pencarian jalur dimulai ketika pengguna menentukan titik awal dan titik tujuan, lalu menekan tombol pencarian rute. Sistem secara otomatis akan mengekstrak nilai koordinat (lintang dan bujur) dari kedua titik tersebut dan mengirimkannya ke modul komputasi

algoritma. Di dalam modul ini, graf representasi jalan kampus Universitas Bengkulu diproses secara simultan menggunakan dua metode komputasi: algoritma Dijkstra dan algoritma A*. Setelah kedua algoritma selesai melakukan kalkulasi dan menemukan urutan simpul optimal, sistem akan mengubah kumpulan data koordinat tersebut menjadi sebuah garis lintasan (*polyline*). Garis lintasan ini kemudian dirender secara dinamis di atas peta *Leaflet.js*, menghubungkan titik awal dan titik tujuan menyusuri topologi jalan kampus yang sebenarnya.



Gambar 1.2 Visualisasi Hasil Pencarian Rute Terpendek pada Peta

4.3 Analisis dan Perbandingan Kinerja Algoritma

Salah satu tujuan utama dari pengembangan sistem ini adalah untuk melakukan analisis komparatif terhadap efisiensi kinerja antara algoritma Dijkstra dan algoritma A* pada studi kasus pemetaan dunia nyata. Pengujian dilakukan dengan menjalankan kedua algoritma pada parameter titik awal dan titik tujuan yang sama di lingkungan kampus Universitas Bengkulu. Indikator kinerja yang diukur meliputi total jarak rute yang dihasilkan, jumlah simpul (*node*) yang dievaluasi selama proses pencarian, serta waktu eksekusi komputasi.

Berdasarkan hasil pengujian pada sistem, diperoleh analisis sebagai berikut:

1. **Akurasi Jarak Tempuh:** Baik algoritma Dijkstra maupun algoritma A* berhasil menemukan rute yang sama persis dan menghasilkan total jarak tempuh yang identik. Hal ini membuktikan bahwa implementasi fungsi heuristik pada algoritma A* bersifat *admissible* (tidak melebihi-lebihkan jarak asli), sehingga tetap mampu menjamin penemuan rute yang mutlak optimal layaknya Dijkstra.

2. **Efisiensi Eksplorasi Simpul:** Terdapat perbedaan yang sangat signifikan pada jumlah simpul yang dievaluasi. Algoritma Dijkstra mengeksplorasi jumlah simpul yang jauh lebih banyak karena prinsip kerjanya yang mengekskansi jaringan ke segala arah secara merata (*blind search*). Sebaliknya, algoritma A* secara drastis mengurangi jumlah simpul yang dievaluasi karena fungsi heuristiknya (jarak garis lurus *Haversine*) berhasil memandu arah pencarian langsung menuju ke titik tujuan secara terfokus.
3. **Waktu Eksekusi (Kecepatan Komputasi):** Sebagai korelasi langsung dari penurunan jumlah simpul yang dievaluasi, algoritma A* mencatatkan waktu eksekusi komputasi (dalam satuan milidetik) yang lebih cepat dibandingkan algoritma Dijkstra.

Dari data analisis tersebut, dapat disimpulkan bahwa untuk penyelesaian masalah pencarian rute terpendek pada graf geografis berskala menengah hingga besar seperti kompleks Universitas Bengkulu, algoritma A* menawarkan tingkat efisiensi komputasi dan manajemen memori yang jauh lebih superior dibandingkan algoritma Dijkstra, tanpa adanya penurunan kualitas akurasi pada rute yang dihasilkan.

4.4 Pengujian Fungsionalitas Sistem

Pengujian fungsionalitas sistem pada proyek ini dilakukan menggunakan metode *Black-Box Testing*. Pendekatan ini bertujuan untuk mengevaluasi respons antarmuka dan memastikan seluruh fitur interaktif berjalan sesuai dengan spesifikasi perancangan tanpa perlu meninjau logika kode internal. Fokus pengujian tahap pertama meliputi validasi input pada menu pilihan (*drop-down*) untuk lokasi asal dan lokasi tujuan. Sistem diuji kemampuannya dalam menangani skenario kesalahan (*error handling*), di mana sistem terbukti secara efektif menolak eksekusi pencarian dan memberikan notifikasi peringatan apabila pengguna memilih titik awal dan tujuan yang sama persis.

Tahap pengujian selanjutnya difokuskan pada fungsionalitas pemetaan visual dan sinkronisasi antarmuka aplikasi. Pengujian ini memverifikasi bahwa garis lintasan rute (*polyline*) dari hasil pencarian sebelumnya dapat terhapus secara otomatis dan diperbarui dengan rute yang baru setiap kali pengguna melakukan permintaan pencarian. Hal ini sangat penting untuk mencegah terjadinya penumpukan grafis di layar yang dapat membingungkan pengguna. Berdasarkan keseluruhan skenario yang telah dilakukan, hasil pengujian menunjukkan bahwa seluruh fungsi antarmuka, validasi input, dan logika pemetaan visual beroperasi secara valid, responsif, dan stabil tanpa adanya temuan galat (*error*) pada sistem.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan seluruh tahapan perancangan, implementasi, pengujian, dan analisis yang telah dijabarkan pada bab-bab sebelumnya, dapat ditarik beberapa kesimpulan utama dari proyek ini. Purwarupa sistem navigasi kampus Universitas Bengkulu berbasis web telah berhasil dikembangkan secara fungsional. Integrasi antara data spasial terbuka dari *OpenStreetMap* dan pustaka antarmuka *Leaflet.js* terbukti efektif dalam menyajikan visualisasi peta digital yang interaktif dan mudah diakses oleh pengguna melalui peramban web (*browser*). Selain itu, implementasi algoritma Dijkstra dan algoritma A* (*A-Star*) pada sistem berjalan dengan sangat baik dan akurat untuk menyelesaikan masalah pencarian rute terpendek (*shortest path problem*) antar fasilitas atau gedung di dalam ruang lingkup area kampus.

Berdasarkan hasil analisis komparatif yang dilakukan pada sistem, algoritma Dijkstra dan algoritma A* terbukti mampu menghasilkan jalur optimal dengan total jarak tempuh yang identik. Namun, algoritma A* menunjukkan kinerja komputasi yang jauh lebih efisien dibandingkan algoritma Dijkstra. Dengan memanfaatkan fungsi heuristik *Haversine*, algoritma A* mampu memandu arah pencarian langsung menuju titik tujuan. Pendekatan terarah ini secara signifikan menekan jumlah simpul (*node*) yang harus dievaluasi sehingga mempercepat waktu eksekusi komputasi sistem tanpa mengorbankan tingkat akurasi pada rute akhir.

5.2 Saran

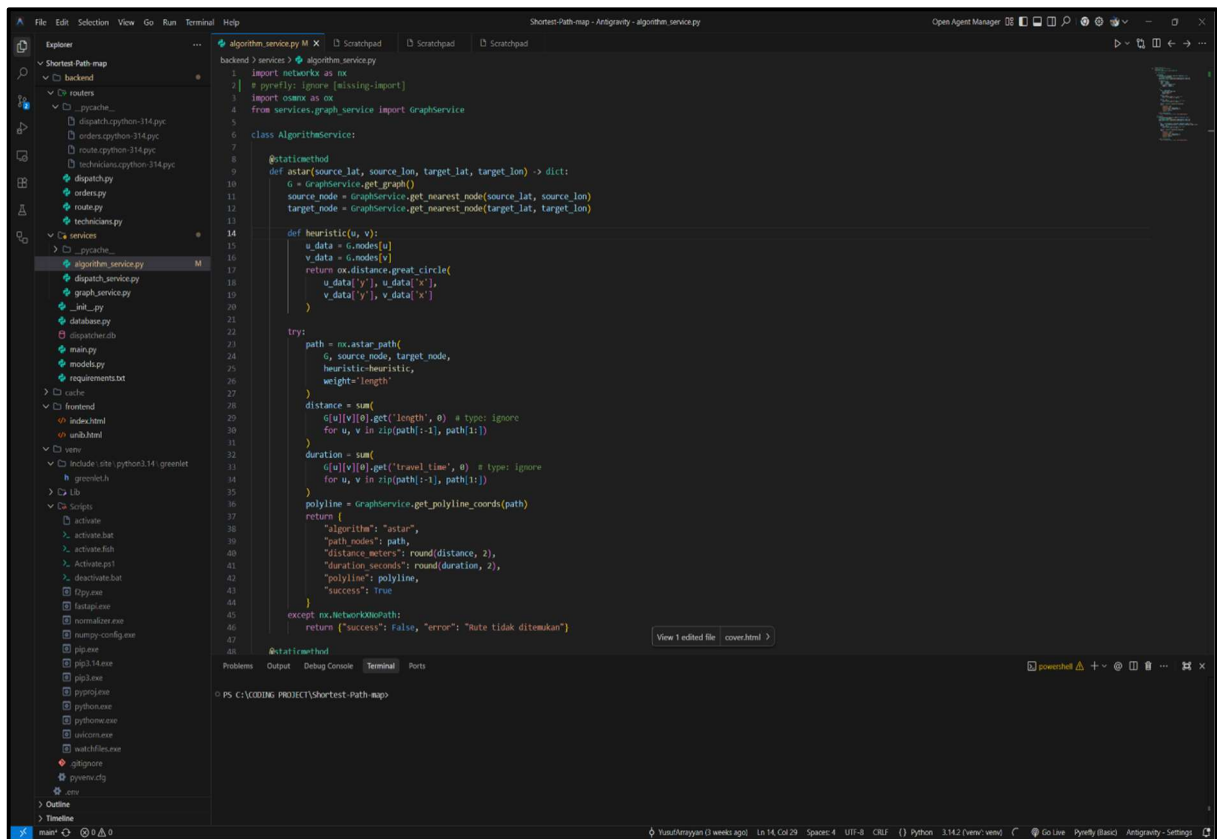
Meskipun purwarupa sistem navigasi kampus ini telah beroperasi dengan baik dan stabil, masih terdapat ruang untuk penyempurnaan di masa mendatang. Salah satu pengembangan utama yang disarankan adalah integrasi fitur *Global Positioning System* (GPS) agar sistem dapat mendeteksi posisi geografis pengguna saat ini secara *real-time* dan menjadikannya sebagai titik awal pencarian rute secara otomatis. Selain itu, arsitektur sistem dapat dikembangkan lebih lanjut menjadi aplikasi *mobile* berbasis Android dan iOS, atau diubah ke dalam format *Progressive Web App* (PWA) agar performa lebih optimal dan responsif.

Pengembangan sistem juga dapat difokuskan pada peningkatan akurasi data pemetaan dan kalkulasi. Sistem dapat diperkaya dengan pembobotan rute dinamis yang mempertimbangkan variabel di lapangan, seperti kepadatan jalan internal, kondisi penutupan gerbang pada jam tertentu, atau rintangan cuaca agar estimasi waktu tempuh menjadi lebih realistis. Terakhir, disarankan untuk melakukan pemetaan mikro pada basis data graf, yang mencakup penambahan titik-titik jalan setapak pedestrian, koridor penghubung antar gedung, hingga fasilitas tangga yang belum terindeks secara mendetail pada penyedia data *OpenStreetMap*.

DAFTAR PUSTAKA

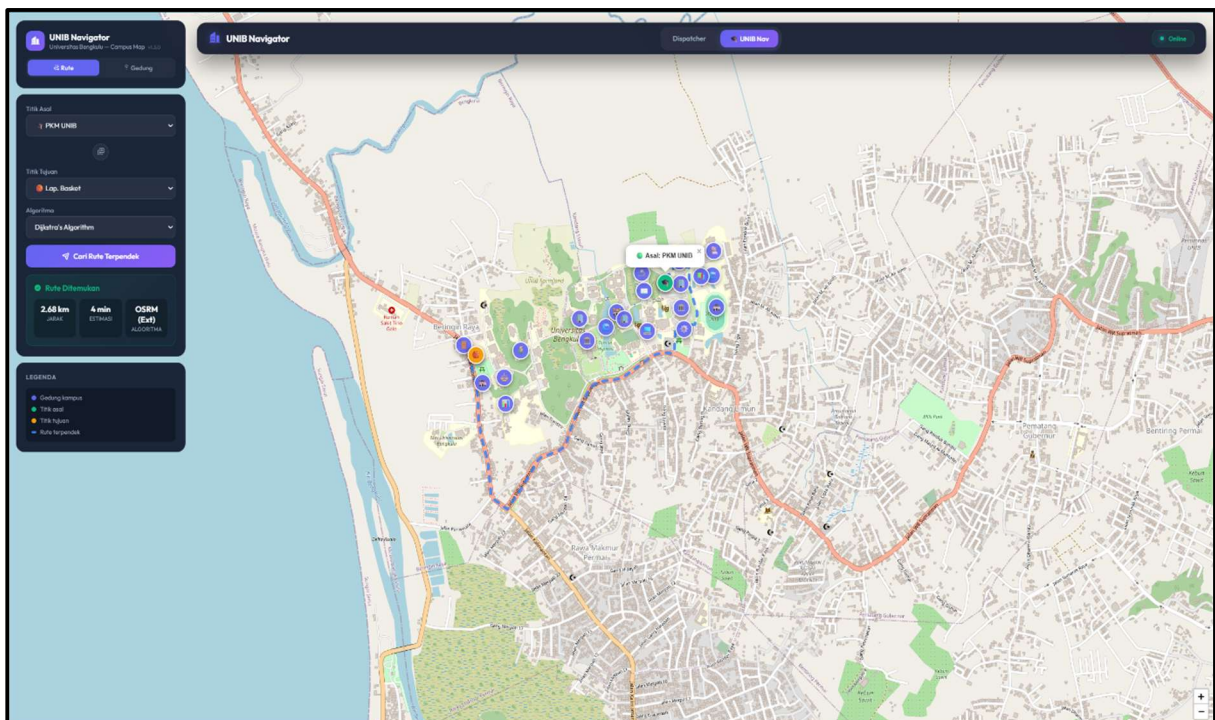
- Andini, M., dkk. (2025). Optimasi pencarian rute terpendek menggunakan algoritma Dijkstra. *JISAMAR (Journal of Information System, Applied, Management, Accounting and Research)*, 9(1).
- Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100-107.
- Longley, P. A., Goodchild, M. F., Maguire, D. J., & Rhind, D. W. (2015). *Geographic Information Systems and Science* (4th ed.). John Wiley & Sons.
- Prasetyo, A. C. (2019). Perbandingan Algoritma Astar dan Dijkstra dalam Penentuan Rute Terdekat. *Sisfotenika*..
- Russell, S. J., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
- Salsabila, N. A., dkk. (2024). Implementasi algoritma Dijkstra dalam penentuan jalur terpendek antara Kampus 1 dan Kampus 2 UIN Gusdur Pekalongan. *Prosiding SANTIKA (Seminar Nasional Tadris Matematika)*.

LAMPIRAN

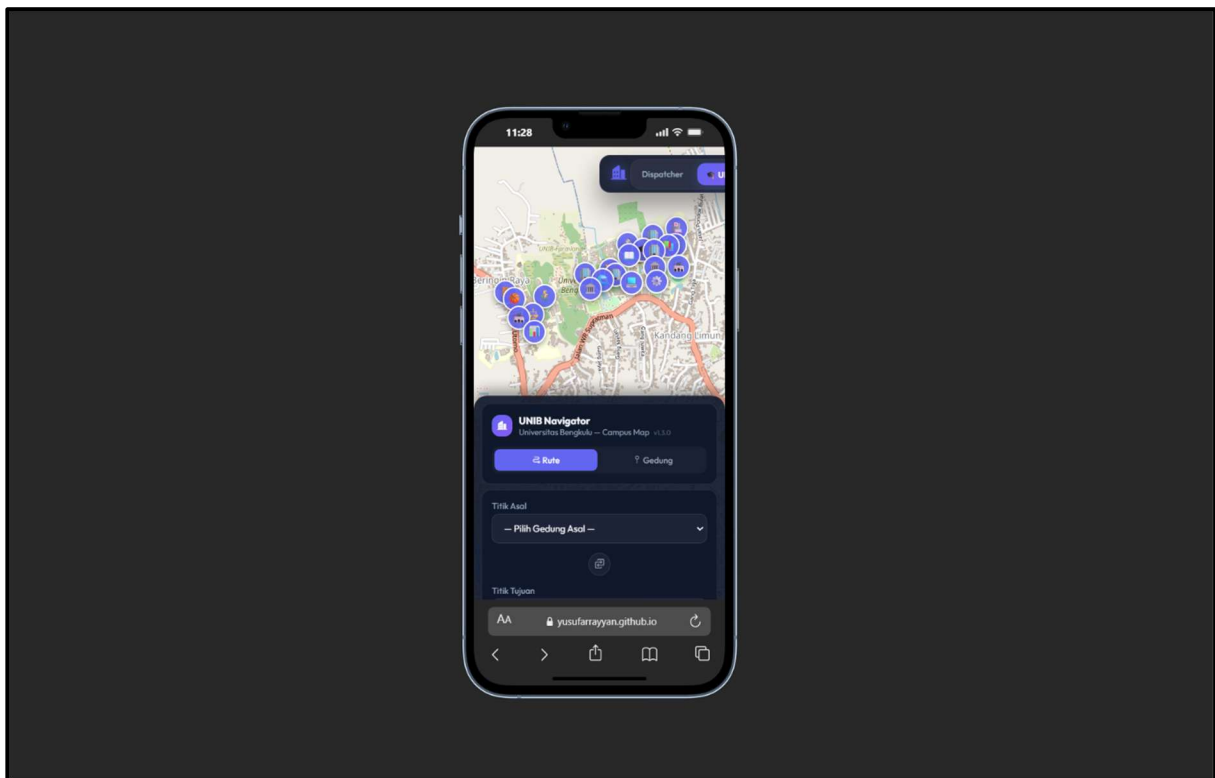


```
1 import networkx as nx
2 # pylint: ignore [missing-import]
3 import osmnx as ox
4 from services.graph_service import GraphService
5
6
7 class AlgorithmService:
8
9     @staticmethod
10     def astar(source_lat, source_lon, target_lat, target_lon) -> dict:
11         g = GraphService.get_graph()
12         source_node = GraphService.get_nearest_node(source_lat, source_lon)
13         target_node = GraphService.get_nearest_node(target_lat, target_lon)
14
15         def heuristic(u, v):
16             u_data = g.nodes[u]
17             v_data = g.nodes[v]
18             return ox.distance.great_circle(
19                 u_data['y'], u_data['x'],
20                 v_data['y'], v_data['x']
21             )
22
23         try:
24             path = nx.astar_path(
25                 g, source_node, target_node,
26                 heuristic=heuristic,
27                 weight='length'
28             )
29             distance = sum(
30                 g[u][v].get('length', 0) * type: ignore
31                 for u, v in zip(path[:-1], path[1:])
32             )
33             duration = sum(
34                 g[u][v].get('travel time', 0) * type: ignore
35                 for u, v in zip(path[:-1], path[1:])
36             )
37             polyline = GraphService.get_polyline_coords(path)
38             return {
39                 "algorithm": "astar",
40                 "path_nodes": path,
41                 "distance_meters": round(distance, 2),
42                 "duration_seconds": round(duration, 2),
43                 "polyline": polyline,
44                 "success": True
45             }
46         except nx.NetworkXNoPath:
47             return {"success": False, "error": "Route tidak ditemukan"}
```

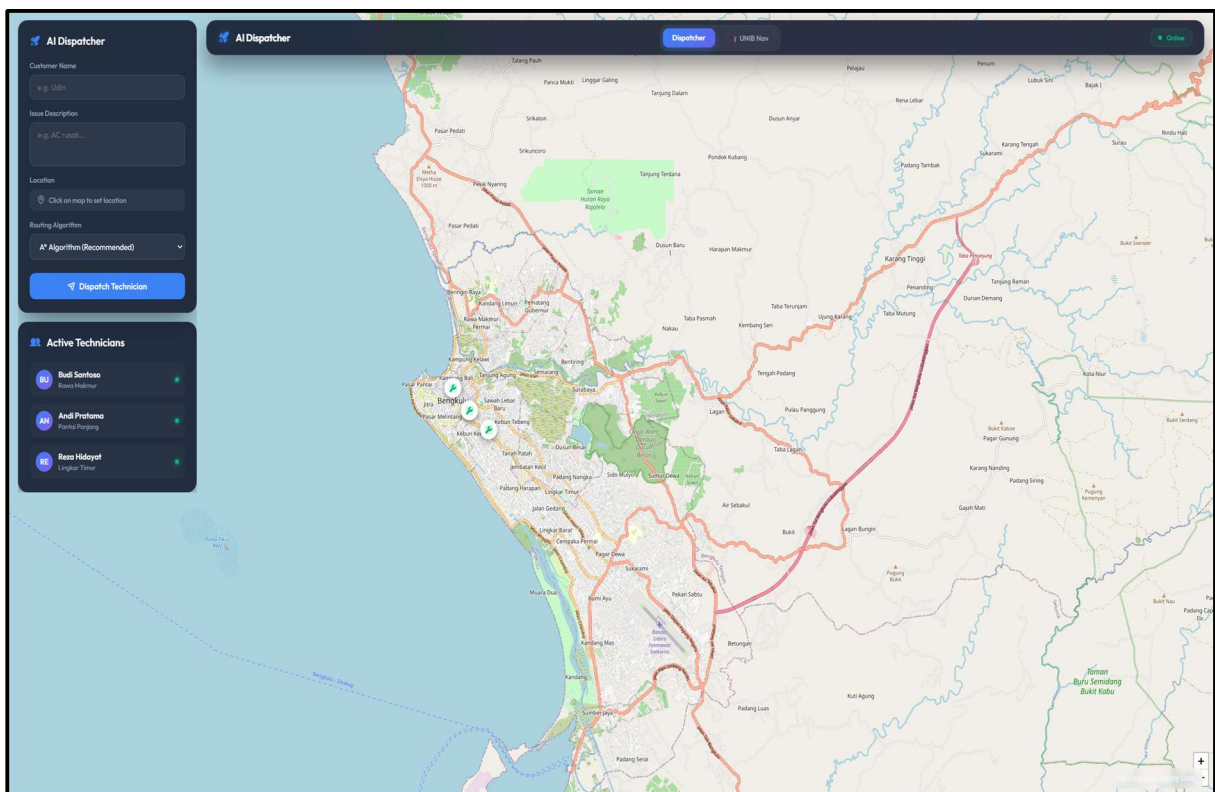
Gambar 2. Source Code *Backend* (Python) untuk Implementasi Algoritma Dijkstra



Gambar 3. Visualisasi Pengujian Pemetaan Route Jarak Jauh Melintasi Area Kampus



Gambar 4. Tampilan Responsif Antarmuka Sistem Navigasi pada Perangkat *Mobile*



Gambar 5. Fitur Tambahan Untuk Menentukan Jarak Berdasar Titik Tertentu.